
Holo 1.0: A Generalist Model that Navigates Virtual Worlds and Completes Hour-Long Objectives

Technical Report

Philon
info@philon.ai

Abstract

Holo 1.0 is an open-source generalist model capable of completing extended missions across 3D open-world video games. Holo 1.0 integrates observation, reasoning, and action into a single model that processes raw 640×360 frames at 12 Hz and outputs keyboard–mouse actions at 21 Hz, engaging explicit reasoning only when necessary. Trained on 81,386 hours of gameplay across three curriculum stages—pre-training, instruction following, and reasoning—Holo 1.0 completes the introductory prologue region of The Witcher 3: Wild Hunt (White Orchard) in 78 minutes—on par with experienced human players (72 minutes) and ahead of novice players (98 minutes)—and the same model generalizes to Starfield, Cyberpunk 2077, and Minecraft, completing 20+ minute objectives in Minecraft despite its first-person voxel rendering and procedurally generated worlds. Code is available at <https://github.com/Philon-AI/holo-1>.

1 Introduction

Creating general-purpose systems that perceive, reason, and act across open-ended worlds is a longstanding goal in AI. Existing work has achieved superhuman performance in constrained domains like Atari, Go, Dota, and StarCraft (Mnih et al., 2015; Silver et al., 2016; Berner et al., 2019; Vinyals et al., 2019), yet these models are trained against a single shaped reward and fail to transfer to the breadth of open-world settings. A separate line of research embeds large vision–language models (VLMs) (Alayrac et al., 2022) in interactive environments via prompting (Wang et al., 2023) or vision–language–action fine-tuning (Brohan et al., 2023; Kim et al., 2024), but such approaches are typically limited to 1 Hz or below, break down on tasks requiring sustained attention, and depend on manually designed skill APIs.

We frame open-world play around four intertwined challenges: *spatio-temporal understanding*; *spatial memory and self-localization* within unknown or procedurally generated worlds; *reasoning and high-level planning* over long horizons; and *low-level control*, the joint generation of discrete and continuous actions. We address all four within a single 31B-parameter model.

Three design choices make this possible: (i) a unified observe–reason–act architecture built on gemma-4-31b (Gemma Team, 2025) that emits keyboard–mouse actions directly in the language space; (ii) a three-stage training curriculum (72,132 h pre-training, 7,112 h instruction-following, 2,142 h reasoning) curated automatically from raw human gameplay; and (iii) a hybrid thinking strategy that invokes reasoning only when needed.

Holo 1.0 is, to our knowledge, the first single model to complete the entire 78-minute White Orchard prologue of The Witcher 3: Wild Hunt, on par with expert human players (72 minutes), while also generalizing to Starfield and Cyberpunk 2077 and completing 20+ minute objectives in Minecraft across its procedurally generated voxel worlds.

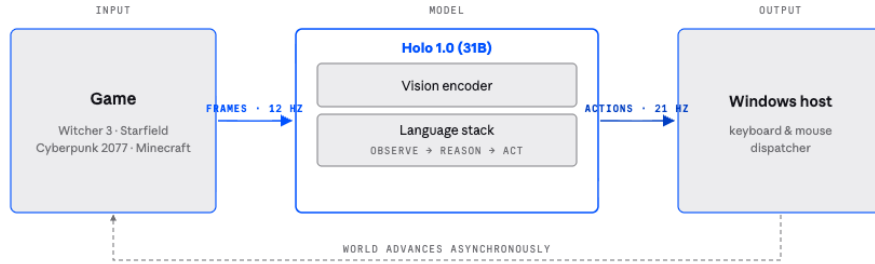


Figure 1: System overview: raw 640×360 frames at 12 Hz $\rightarrow$ Holo 1.0 (31B) $\rightarrow$ keyboard–mouse actions at 21 Hz, with the hybrid thinking branch invoked only at transitions.

2 Related Work

Closed-domain models. Reinforcement-learning models for Atari, Go, Dota II, StarCraft II, and Gran Turismo (Mnih et al., 2015; Silver et al., 2016; Berner et al., 2019; Vinyals et al., 2019; Wurman et al., 2022) achieve super-human play but require dense reward shaping, perfect simulators, and large-scale parallel rollouts. None of these assumptions hold for commercial 3D open worlds, which expose only screen pixels, run asynchronously, and lack programmatic reward signals.

VLM and VLA models. Prompt-based VLMs act through hand-coded skill APIs (Wang et al., 2023) and are bottlenecked by API latency on the order of seconds, far too slow for interactive play. Vision–language–action models trained on robotic or GUI trajectories (Brohan et al., 2023; Kim et al., 2024) show that VLMs can be turned into closed-loop controllers, but existing game-domain variants typically rely on code-format actions and short-horizon evaluation. Holo 1.0 differs by (i) representing keyboard–mouse actions directly as text in the existing vocabulary, (ii) processing frames at 12 Hz and emitting actions at 21 Hz via action chunking, and (iii) evaluating on extended missions rather than short tasks.

3 Environment

We use The Witcher 3: Wild Hunt as our primary testbed. It is a third-person open-world ARPG that combines large-scale 3D exploration (running, riding, climbing, swimming) with real-time combat, a substantial 2D GUI for menus, inventory, and quest journal, varied quest and puzzle mechanics, and a multi-hour main storyline. Crucially, the game exposes no API: the model must receive raw screen pixels and emit keyboard–mouse events, and the world continues to evolve while the model is computing. These constraints rule out classical RL and force a fully closed-loop, pixel-in / action-out design.

For cross-game evaluation we use three additional commercial 3D titles spanning distinct control and visual paradigms: Starfield, a sci-fi RPG with mixed third-/first-person exploration of planets and space and heavy menu interaction; Cyberpunk 2077, a first-person open-world action RPG set in a dense futuristic city with gunplay, melee, and hacking-based combat; and Minecraft, a first-person voxel-based sandbox with procedurally generated worlds, open-ended objectives, and crafting and survival mechanics. Minecraft is the most distant from our primary testbed along every relevant axis—it replaces photorealistic rendering with block-based voxels, swaps the third-person camera for first-person, introduces a hotbar-plus-crafting-grid inventory system absent from any other title in our mixture, and, unlike the hand-authored maps of The Witcher 3, Starfield, and Cyberpunk 2077, every Minecraft world is randomly generated from a seed, so no two evaluation runs share the same terrain, biomes, or resource layout.

4 The Holo 1.0 Model

Holo 1.0 builds on gemma-4-31b and reuses its tokenizer and architecture without modification, augmenting it only through a structured action format.

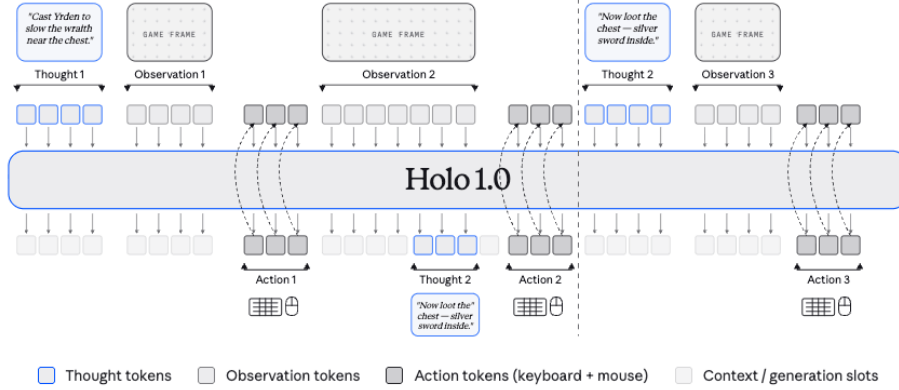


Figure 2: Holo 1.0 model architecture: gemma-4-31b backbone with vision encoder, language stack, and the hybrid-thinking gate that decides between a reasoning trace and a direct action at each step.

Factorization. At each step t , the model first decides whether to emit a new reasoning trace r_t (otherwise $r_t = \emptyset$), then emits an action a_t :

$$\pi_{\theta}(a_t, r_t \mid o_{\leq t}, r_{< t}, a_{< t}) = \pi_{\theta}(a_t \mid o_{\leq t}, r_{\leq t}, a_{< t}) \pi_{\theta}(r_t \mid o_{\leq t}, r_{< t}, a_{< t}). \quad (1)$$

Observation space. Each frame is resized to 640×360 and consumed at 12 Hz (every ≈ 83 ms), matching human visual reaction time. History is encoded as an interleaved multi-turn dialogue of past observations, reasoning, and actions.

Hybrid thinking. A special token $\langle | \text{thought_start} | \rangle$ or $\langle | \text{action_start} | \rangle$ as the first generated token determines whether the model enters the thinking mode. Explicit reasoning is reserved for transitions—when the plan becomes invalid or a subtask is finished—and skipped otherwise. This avoids paying the reasoning latency tax at every step.

Keyboard–mouse action format. Rather than adding action heads or code-style function calls, we represent each action as a semicolon-separated string in the language space:

$$\underbrace{\Delta X \ \Delta Y \ \Delta Z}_{\text{mouse}} ; \underbrace{K_1 ; K_2 ; K_3 ; K_4 ; K_5 ; K_6}_{\text{key chunks}}.$$

Mouse displacement $(\Delta X, \Delta Y) \in (-1000, 1000)^2$ and scroll $\Delta Z \in [-5, 5]$ are executed smoothly over a ≈ 286 ms interval. The six K_i are action chunks of ≈ 48 ms each, giving an effective 21 Hz interaction frequency. Each chunk lists up to four held keys; keys shared across consecutive chunks remain held without re-pressing (action chunking; Zhao et al., 2023). This format faithfully reproduces long presses, key combos, rapid clicking, and drag-and-drop without modifying the base model. A concrete example—dashing right toward a chest, then opening it—is "92 0 0 ; Shift W ; Shift W ; Shift W ; F W ; F W ; F".

Memory. During inference a sliding window of up to 20 recent observation–action pairs serves as short-term memory, while the most recent reasoning trace is preserved across the window flush as long-term memory.

5 Data Curation and Training Recipe

Holo 1.0 is trained in three stages on a mixture of games—The Witcher 3, Starfield, Cyberpunk 2077, and Minecraft—in all three stages. The pre-training corpus consists of raw gameplay videos sourced

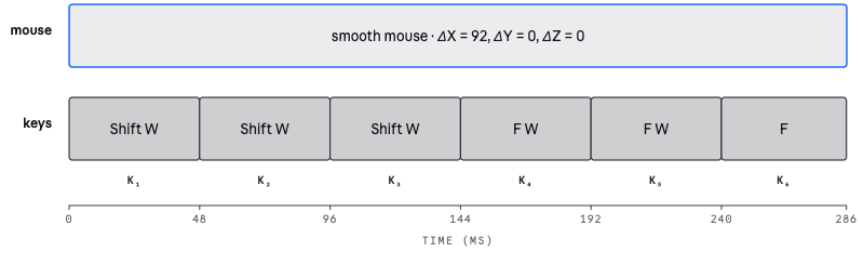


Figure 3: Timing of one ≈ 286 ms action sequence: smooth mouse displacement ($\Delta X, \Delta Y, \Delta Z$) over the full interval, overlaid with six ≈ 48 ms key chunks $K_1, \dots, K_6$ (21 Hz effective rate). Keys held across consecutive chunks are not re-pressed. Annotated with the “dash right toward a chest, then open it” example.

from the public internet, while the smaller instruction-following and reasoning datasets are recorded by contractors on capture rigs that synchronize video with keyboard–mouse events.

Stage 1: Pre-training (Holo 1.0-Base). Unlike prior approaches that rely on large-scale instruction–action data, we expose the model to diverse in-game dynamics using image–action pairs. Because the source videos lack keyboard–mouse traces, a separately trained inverse-dynamics model (Baker et al., 2022) labels each frame transition with the action that most likely produced it. Rule-based filtering then removes 95% of idle actions and camera-jitter clips, leaving 72,132 h of gameplay. We mix in $\approx 20\%$ multimodal web data to preserve the base model’s perception and reasoning, plus a small held-out instruction-following slice. Pre-training already yields Holo 1.0-Base, which exhibits an emergent progression of capabilities with scale: simple object interaction emerges within $\approx 1,700$ h, basic combat and GUI manipulation become reliable at $\approx 10,000$ h, and game-mechanism recognition and robust navigation require $\geq 20,000$ h.



Figure 4: Emergent capabilities during Stage 1 pre-training: benchmark success rate vs. training hours, with dashed thresholds at $\approx 1,700$ h (simple object interaction), $\approx 10,000$ h (basic combat and GUI), and $\geq 20,000$ h (game mechanics and robust navigation).

Stage 2: Instruction following (Holo 1.0-Instruct). We collect 165 h of human annotation tagging gameplay clips with one of 38 activity categories, train a gemma-4-2b classifier on these labels, and use it to auto-label all raw data. Transition points between adjacent labels mark task boundaries; around each transition we extract a 4-second snippet and prompt GPT-4.1 to write a context-aware instruction conditioned on the predicted category, with GPT-4.1 also acting as a verifier that discards inconsistent samples. After action filtering this yields 7,112 h of high-quality instruction–image–action triples, mixed with web data at the same ratio as pre-training.

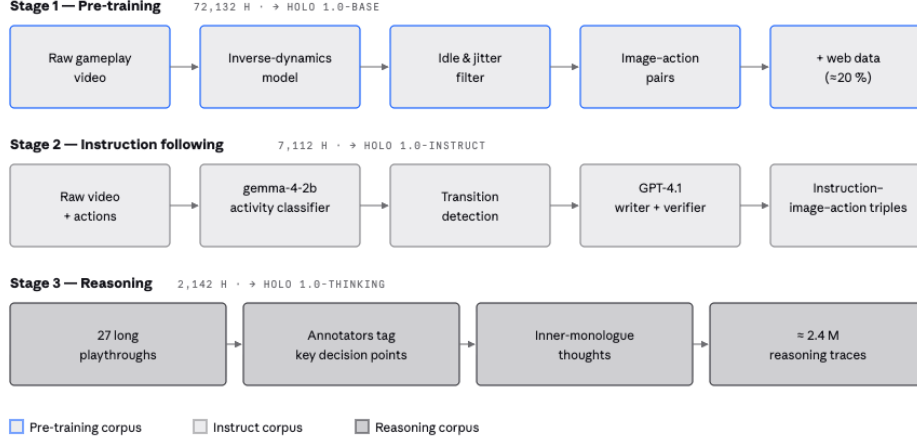


Figure 5: Data curation pipeline across the three stages: raw gameplay video $\rightarrow$ inverse-dynamics action labels (Stage 1); gemma-4-2b activity classifier $\rightarrow$ transition detection $\rightarrow$ GPT-4.1 instruction writer and verifier (Stage 2); annotator-written inner-monologue thoughts at key decision points (Stage 3).

Stage 3: Reasoning (Holo 1.0-Thinking). For 27 long-form playthroughs across The Witcher 3, Starfield, Cyberpunk 2077, and Minecraft (≈ 80 h each), annotators identify key decision points and write concise first-person inner-monologue thoughts. This produces 2,142 h of data containing ≈ 2.4 M reasoning traces (3.2 s average interval; 37.4 ± 11.7 tokens per thought). We do not apply action filtering for this stage, so the model learns to wait at critical decision points. Fine-tuning Holo 1.0-Instruct on this dataset yields Holo 1.0-Thinking, the autonomous model evaluated in Section 8.

Training details. All stages use TorchTitan (Liang et al., 2024) with AdamW (Loshchilov and Hutter, 2019), LR 2×10^{-5} (LLM) and 7×10^{-6} (ViT), batch sequence length 32K, on 256 H100 GPUs. The history setting (20-frame multi-turn input) is trained for 3 epochs in pre-training. The ViT is frozen for the reasoning stage.

6 Inference

The control loop runs on two machines: a Windows host running the game and a remote inference server, connected by TCP. The host streams screenshots to the server at 12 Hz and dispatches keyboard–mouse events as action chunks arrive at 21 Hz; action sequences (≈ 286 ms each) are emitted back-to-back, so each inference cycle must complete within ≈ 286 ms to keep the chunk stream uninterrupted. A naive deployment of a 31B model takes roughly 3.7 s per cycle, more than $12\times$ too slow. We combine five families of optimizations:

- *Streaming output.* Action chunks are dispatched as soon as their terminating semicolon is emitted, so the system only needs to sustain a ≥ 21 Hz chunk *throughput* (≈ 48 ms per chunk on average) rather than producing each full ≈ 286 ms sequence within a single 286 ms wall-clock window.
- *KV-cache reuse.* In the history setting we use StreamingLLM-style attention sinks (Xiao et al., 2024) to maximize KV-cache reuse when the oldest turn is evicted from the window.
- *Tensor parallelism* across multiple GPUs (Shoeybi et al., 2019).
- *LLM quantization* of the language stack.
- *Speculative decoding* for action tokens (Leviathan et al., 2023).

Together, plus residual infrastructure-level changes, these yield 129.8 ms end-to-end latency per cycle—a $28.5\times$ speedup over the baseline—comfortably within the ≈ 286 ms budget.

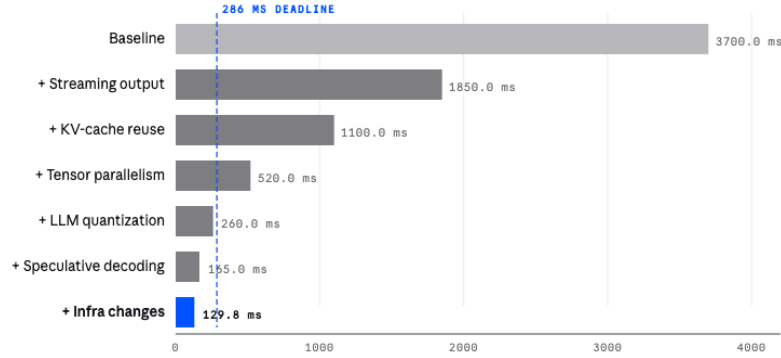


Figure 6: Inference latency waterfall: 3,700 ms baseline $\rightarrow$ 129.8 ms (28.5 $\times$ speedup), with per-optimization contributions (streaming output, KV-cache reuse, tensor parallelism, LLM quantization, speculative decoding, residual infrastructure). Dashed line marks the $\approx$ 286 ms per-cycle deadline.

7 Benchmark

We construct a benchmark of 141 language-conditioned tasks in four categories (Collection, Combat, NPC Interaction, Puzzle) at three difficulties: *simple* (short-horizon, in-distribution, ≤ 10 s except combat), *hard* (3D spatial reasoning, distractors, vertical traversal, elite enemies), and *unseen* (new regions—Velen and Skellige Isles—with novel objects, NPCs, and enemies; puzzles excluded). We compare against prompt-based VLMs (GPT-5, Gemini 3 Pro) with a skill-API action space; those baselines pause the game during inference, while Holo 1.0 runs without pausing.



Figure 7: Example tasks from the 141-task benchmark: rows = difficulty (*simple*, *hard*, *unseen*); columns = category (Collection, Combat, NPC Interaction, Puzzle). Unseen tasks are drawn from Velen and the Skellige Isles.

8 Experimental Results

Scaling. On the first epoch of pre-training, the loss of a smaller gemma-4 variant continues to decline across the full 72,132 h of data while its benchmark success rate degrades—a sign of insufficient capacity. The 31B model improves monotonically on both, motivating our choice of backbone and suggesting further headroom from scale. Capabilities emerge at different rates: simple object interaction ($< 1,700$ h), basic combat and GUI ($\approx 10,000$ h), and game mechanics and robust navigation ($> 20,000$ h).

Short-horizon instruction following. On simple benchmark tasks Holo 1.0-Instruct exceeds 80% success in all four categories, a 61% absolute improvement over Holo 1.0-Base and a substantial gain over all VLM baselines (Table 1). The instruct model corrects behavioral lapses of the base model (e.g., neglecting to loot bodies after combat), and the improvement generalizes to unseen tasks in Velen and the Skellige Isles.

Table 1: Success rate (%) on the 141-task benchmark, averaged over the four categories (Collection, Combat, NPC Interaction, Puzzle) within each difficulty tier. Prompt-based VLM baselines use a skill-API action space and pause the game during inference; Holo 1.0 runs without pausing. Best Holo result per column in **bold**.

Model	Simple	Hard	Unseen	Overall
GPT-5	47.3	24.1	21.8	31.1
Gemini 3 Pro	49.6	26.4	23.5	33.2
Holo 1.0-Base	23.2	11.5	9.4	14.7
Holo 1.0-Instruct	84.2	52.6	41.3	59.4
Holo 1.0-Thinking	88.1	67.8	56.2	70.7

Hours-long autonomy: White Orchard prologue. We divide the prologue into five sequential subtasks—(1) reach White Orchard and rendezvous with Vesemir at the inn, (2) investigate the griffin attack site and track the beast, (3) gather griffin-hunt ingredients (including the underwater Buckthorn herb), (4) defeat the griffin, and (5) return to the Nilfgaardian garrison to meet Captain Peter Saar Gwynleve and depart for Velen—and run each model three times end-to-end without pausing the game (Table 2). Holo 1.0-Thinking with history succeeds on 14/15 attempts and completes the entire prologue in **78 minutes**, on par with experienced human players (72 min average) and ahead of novice players (98 min). The single failure is a fall off a ledge from which the model failed to invoke its quest-guidance reasoning and timed out. Without explicit reasoning, instruct models lose track of the main objective when distracted by side activities or enemies.

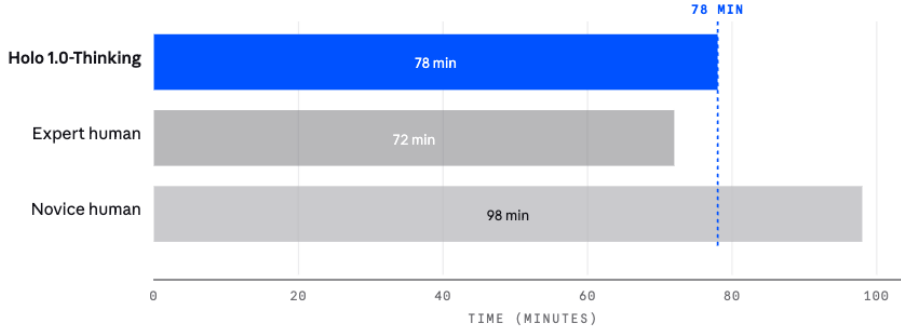


Figure 8: White Orchard prologue: timeline of the five sequential subtasks for Holo 1.0-Thinking (78 min), with reference bars for expert human players (72 min) and novice players (98 min). Subtask boundaries: (1) rendezvous with Vesemir, (2) investigate the griffin attack site and track the beast, (3) gather ingredients, (4) defeat the griffin, (5) meet Captain Peter Saar Gwynleve at the Nilfgaardian garrison.

Table 2: Success rate on the five sequential subtasks of the White Orchard prologue, three runs each.

Model	Overall	Task 1	Task 2	Task 3	Task 4	Task 5
Holo 1.0-Instruct	66.7%	2/3	3/3	2/3	2/3	1/3
Holo 1.0-Thinking	93.3%	3/3	2/3	3/3	3/3	3/3

Reasoning quality. Error analysis on the full mission shows the model generates 593 reasoning instances with an 8.8% error rate. The dominant error modes are perceptual misrepresentation, premature completion, causal misjudgment, and self-contradiction.

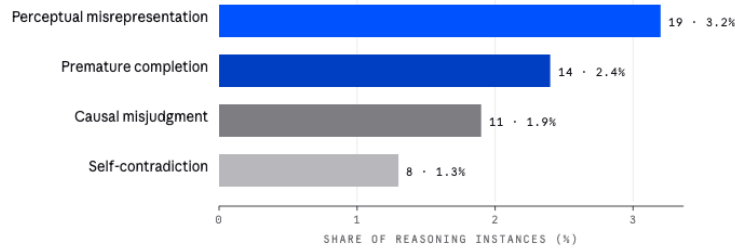


Figure 9: Breakdown of reasoning errors among the 593 reasoning instances on the White Orchard prologue (8.8% overall error rate): perceptual misrepresentation, premature completion, causal misjudgment, and self-contradiction.

Reasoning OOD: Velen and Novigrad. The Velen and Novigrad main-story arcs together total 20–25 h of human play and lie outside the reasoning-stage data (they appear in pre-training and instruction-following data but are excluded from the reasoning data). End-to-end completion of either full arc remains out of reach for Holo 1.0-Thinking. On shorter missions within these regions, however, the model succeeds despite the distribution shift, with inefficiencies traceable to five identifiable causes: insufficient proactive fast-travel, weak minimap use, no proactive health recovery, limited 4-second memory when multiple quest markers compete, and combat skill coordination.

Fully OOD: Skellige Isles. On the Skellige Isles (entirely new region, NPCs, and terrain), Holo 1.0 sustains near-in-domain efficiency on short traversal and exploration segments, but does not reliably complete the full route to Kaer Trolde keep; performance degrades as the distribution shift compounds over longer horizons.

Starfield. The same Holo 1.0-Thinking model completes shorter cross-game objectives in Starfield’s planet-side exploration and ship-based segments. Menu and inventory interactions transfer cleanly from The Witcher 3; ship piloting and zero-g controls account for the dominant residual failures.

Cyberpunk 2077. On Cyberpunk 2077 open-world traversal and mid-tier combat encounters, the model navigates Night City and reliably engages basic enemies on foot and in vehicles. High-intensity firefights and boss encounters with demanding aim and reaction timing remain the dominant failure mode, consistent with the limits of the 21 Hz action rate on frame-precise shooter mechanics.

Minecraft. The same Holo 1.0-Thinking model completes 20+ minute objectives in Minecraft despite its first-person voxel rendering and substantially different control paradigm from The Witcher 3. Basic 3D navigation, combat, and inventory interaction transfer cleanly across games in the mixture; Minecraft-specific affordances such as block placement physics and the hotbar account for the dominant residual failures. The pattern indicates that core 3D navigation and 2D GUI skills generalize across visual domains, while game-specific combat and interaction conventions remain harder to share. Among the four titles in our mixture Minecraft is the strongest test of cross-domain transfer: its voxel rendering, first-person camera, crafting-grid inventory, and randomly generated world are each unique to it, so successful play implies that the learned policies are not specific to photorealistic open-world RPGs but extend to fundamentally different visual and control regimes.

9 Discussion

Holo 1.0 demonstrates that synchronized human gameplay and 256 H100 GPUs are sufficient to turn an open-source 31B VLM into a generalist model that completes extended missions across multiple



Figure 10: Representative frames from the four games in the training mixture—The Witcher 3, Starfield, Cyberpunk 2077, Minecraft—illustrating the visual and control gap that the same Holo 1.0-Thinking model spans (photorealistic open-world action RPG vs. first-person procedurally generated voxel sandbox).

commercial 3D open worlds within a single model. Limitations and natural extensions are: (i) scaling pre-training and reasoning data; (ii) adding parametric memory to scale knowledge capacity without inflating compute (Lample et al., 2019); (iii) strengthening spatio-temporal understanding to better integrate visual evidence across longer horizons; (iv) introducing a dedicated spatial memory module for persistent self-localization and map building in unknown or procedurally generated worlds; (v) layering online reinforcement learning on top of the offline-trained base to surpass human-level efficiency; and (vi) further reducing inference latency to enable both faster control and faster RL rollouts.

References

- Alayrac, J.-B., Donahue, J., Luc, P., et al. (2022). Flamingo: A visual language model for few-shot learning. In *NeurIPS*.
- Baker, B., Akkaya, I., Zhokhov, P., et al. (2022). Video PreTraining (VPT): Learning to act by watching unlabeled online videos. In *NeurIPS*.
- Berner, C., Brockman, G., Chan, B., et al. (2019). Dota 2 with large scale deep reinforcement learning (OpenAI Five). *arXiv:1912.06680*.
- Brohan, A., Brown, N., Carbajal, J., et al. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv:2307.15818*.
- Gemma Team (2025). Gemma 3 technical report. *arXiv:2503.19786*.
- Kim, M. J., Pertsch, K., Karamcheti, S., et al. (2024). OpenVLA: An open-source vision-language-action model. *arXiv:2406.09246*.
- Lample, G., Sablayrolles, A., Ranzato, M., Denoyer, L., and Jégou, H. (2019). Large memory layers with product keys. In *NeurIPS*.
- Leviathan, Y., Kalman, M., and Matias, Y. (2023). Fast inference from transformers via speculative decoding. In *ICML*.
- Liang, W., Liu, H., Wang, C., et al. (2024). TorchTitan: One-stop PyTorch native solution for production-ready LLM pre-training. *arXiv:2410.06511*.
- Loshchilov, I., and Hutter, F. (2019). Decoupled weight decay regularization (AdamW). In *ICLR*.
- Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Shoeybi, M., Patwary, M., Puri, R., et al. (2019). Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv:1909.08053*.
- Silver, D., Huang, A., Maddison, C. J., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning (AlphaStar). *Nature*, 575(7782):350–354.
- Wang, G., Xie, Y., Jiang, Y., et al. (2023). Voyager: An open-ended embodied agent with large language models. *arXiv:2305.16291*.
- Wurman, P. R., Barrett, S., Kawamoto, K., et al. (2022). Outracing champion Gran Turismo drivers with deep reinforcement learning (GT Sophy). *Nature*, 602(7896):223–228.

- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. (2024). Efficient streaming language models with attention sinks (StreamingLLM). In *ICLR*.
- Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware (ACT / action chunking). In *RSS*.